

AlphaZero for Sequential Navigation

Diagnosing Scaling Failures in the Doors Problem

Milton Lin

March 2026

Outline

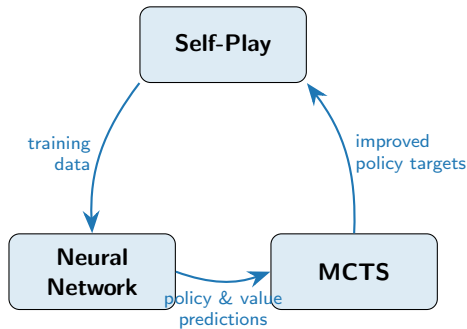
- 1 Background: AlphaZero
- 2 The Doors Problem
- 3 The $D=14$ Failure
- 4 Systematic Small-Scale Verification
- 5 Diagnosis and Conclusions

AlphaZero in One Slide

Three components in a virtuous cycle:

- 1 **Self-Play:** Current network plays games, collecting trajectories (s_t, π_t, z_t)
- 2 **Neural Network:** Learns to predict MCTS policy π and game outcome v from state s
- 3 **MCTS:** Tree search guided by the network produces improved policy targets

Silver et al., *Science*, 2018



Monte Carlo Tree Search (MCTS)

Action selection via UCB:

$$\text{UCB}(s, a) = \bar{Q}(s, a) + c \cdot \pi_{\theta}(a|s) \cdot \frac{\sqrt{N(s)}}{1 + N(s, a)}$$

Four phases per simulation:

- 1 **Select:** Follow UCB down the tree
- 2 **Expand:** Add new leaf, query network for (π, v)
- 3 **Backup:** Propagate v up the path
- 4 **Return:** Root visit counts $\rightarrow$ policy target

Key insight:

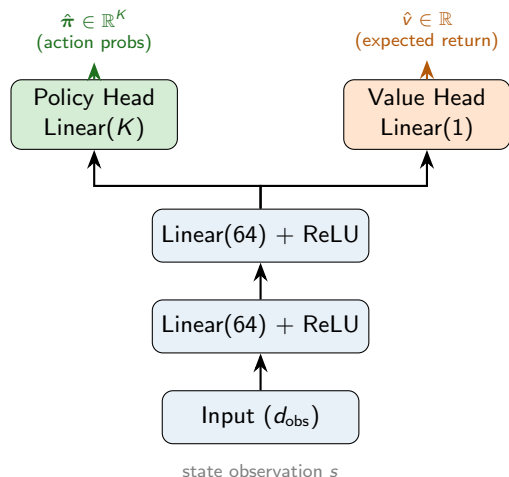
The visit-count distribution at the root becomes the **training target** for the policy head.

Exploration: Dirichlet noise added at root:

$$\pi'(a) = (1 - \epsilon) \pi_{\theta}(a) + \epsilon \eta_a$$

where $\eta \sim \text{Dir}(\alpha)$

The Neural Network



Input: state observation $s \in \mathbb{R}^{d_{\text{obs}}}$

Outputs:

- $\hat{\pi}(\cdot|s)$: probability over K actions
- $\hat{v}(s)$: predicted game return (scalar)

Training targets (from self-play):

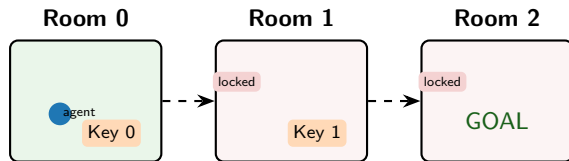
- π_{MCTS} : MCTS visit-count distribution
- z : actual discounted return of the game

Loss:

$$\mathcal{L} = \underbrace{(\hat{v}(s) - z)^2}_{\text{value loss: predict game outcome}} + 2 \cdot \underbrace{\text{CE}(\hat{\pi}, \pi_{\text{MCTS}})}_{\text{policy loss: match MCTS policy}}$$

Adam, $\text{lr} = 3 \times 10^{-4}$, grad clip at norm 1.0

The Doors Problem



Example: $D = 3$ rooms

Actions:

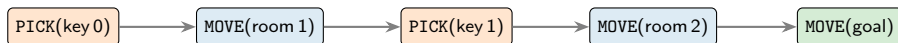
- $\text{MOVE_TO}(\ell)$ – move to location
- $\text{PICK}(k)$ – pick up key k
- NOOP – do nothing

Rewards: $-0.01/\text{step}$, $+0.10/\text{unlock}$

Observation:

One-hot location + unlock status + key availability

Optimal plan for $D=3$ (5 steps):



General pattern: $(\text{PICK} \rightarrow \text{MOVE})^{D-1} \rightarrow \text{MOVE}(\text{goal})$. Optimal length = $2(D-1) + 1$ steps.

Why Doors is Hard for AlphaZero

- ① **No action masking:** $\sim 75\text{--}80\%$ of actions are **silent no-ops**
Moving to a locked room or picking an unavailable key does nothing – no negative signal, just wasted time.
- ② **Long sequential dependencies:** Must execute key $\rightarrow$ unlock chain in strict order
 $D = 14$ requires a 27-step optimal plan. Every step must be correct.
- ③ **Combinatorial action space:** $K = 4D$ total actions, but only $\sim 5\text{--}6$ useful at any state
MCTS must discover the correct sequence by chance before the network can learn.

Core challenge: MCTS must explore deeply enough to see the key $\rightarrow$ unlock reward chain (depth ≥ 2) to produce useful policy targets.

Problem Dimensions

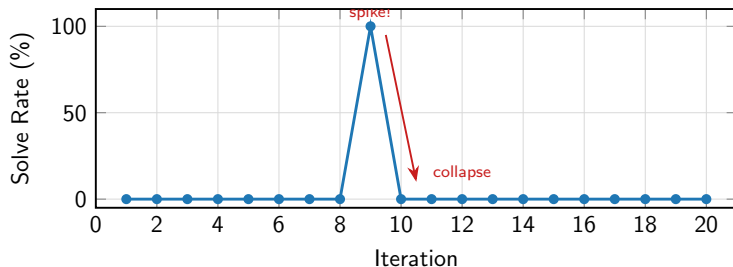
D	obs	actions (K)	opt. steps	horizon	net params	state space
2	9	8	3	15	1,225	48
3	14	12	5	25	1,805	288
4	19	16	7	35	2,829	1,536
5	24	20	9	45	4,437	7,680
6	29	24	11	55	6,405	36,864
14	69	56	27	135	$\sim 40\text{K}$	$\sim 10^{10}$

State space grows **exponentially**; action count grows **linearly** with D .

All experiments: `locs_per_room=3`, $T = 1.0$, $c = 1.5$, Dirichlet noise, no action masking, no gating.

The Motivating Failure: $D=14$ Spike-and-Collapse

Configuration: $D = 14$, sims= 120, games= 20, 20 iterations



- Iterations 1–8: 0% solve rate. Policy loss ≈ 3.9 (near $\ln 56 = 4.03$, i.e. uniform).
- Iteration 9: sudden spike to 100%.
- Iteration 10+: immediate collapse, never recovers.

Hypothesis: Data Starvation

Prior successful D=14 runs used games=50.

This run used games=20 – $2.5\times$ less data.

The numbers:

- games= 20, horizon= 135
- $\Rightarrow \sim 2,700$ examples/iteration
- Network has $\sim 40K$ parameters
- Buffer at iter 10: $\sim 27K$ examples
- Ratio: $\sim 0.7\times$ params

The mechanism:

- Network overfits each small batch
- Destroys previously learned weights
- Spike = one lucky iteration
- Collapse = catastrophic forgetting

Question: Can we reproduce this failure at small scale and map the exact thresholds?

Experimental Design

13 runs across four experiment groups:

Group	Question	Sweep variable	Fixed
A	Does the code work?	$D \in \{2, 3\}$	sims=25, games=20
B	How much search?	$\text{sims} \in \{5, 10, 25, 50\}$	$D = 4$, games=20
C	How much data?	$\text{games} \in \{5, 10, 20, 40\}$	$D = 4$, sims=25
D	How does difficulty scale?	$D \in \{2, 3, 4, 5, 6\}$	sims=25, games=20

All experiments held constant:

$T=1.0$, $c=1.5$, $\text{Dir}(\alpha=0.25, \epsilon=0.4)$, replay buffer = 20 iters, no gating, no masking, 8 parallel workers.

Experiment A: Sanity Check

Settings: sims= 25, games= 20, 15 iterations.

D	First solve	Optimal by	Stable?
2	iter 1	iter 4	Yes – 100% all 15 iterations
3	iter 3	iter 7	Yes – 100% from iter 3 onward

Verdict: The code is correct.

$D=2$ solves from iteration 1 (MCTS alone finds the 3-step plan with a random network).

$D=3$ converges cleanly by iteration 7.

Experiment B: MCTS Simulation Sweep ($D=4$)

sims	sims/ K	First solve	Outcome
5	0.31	iter 19	Spike-collapse Brief spike then anti-learning
10	0.63	never	Dead 0% all 30 iterations
25	1.56	iter 2	Stable Optimal by iter 13
50	3.12	iter 1	Stable MCTS alone solves

- **Sharp threshold** between $\text{sims}/K = 0.63$ (dead) and 1.56 (stable)
- **Counterintuitive:** $\text{sims}=10$ is *worse* than $\text{sims}=5$
 - At $\text{sims}=5$: tree occasionally stumbles into rewarding branch
 - At $\text{sims}=10$: visits spread uniformly but not enough to differentiate $\rightarrow$ maximally uninformative targets

Why the MCTS Threshold is Sharp

Below threshold ($\text{sims}/K < 1$):

- With random network, visits spread roughly uniformly across K actions
- Most actions get 0 or 1 visit
- Visit counts are **noise**
- Network trains on garbage
- No learning signal at all

Above threshold ($\text{sims}/K \geq 1.5$):

- Enough visits for UCB to **differentiate** good from bad actions
- Tree reaches depth ≥ 2 : sees key→unlock reward chain
- Visit counts carry **real signal**
- Network improves, MCTS improves
- Virtuous cycle engages

The transition from “random noise” to “useful signal” happens in a **narrow band** around $\text{sims}/K \approx 1$.

Experiment C: Data Volume Sweep ($D=4$, sims= 25)

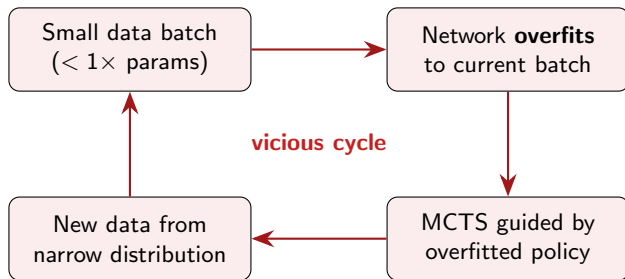
games	ex./iter	buffer/params	First solve	Outcome
5	~ 175	$0.6\times$	iter 3	Spike-and-collapse (10 collapses in 30 iters)
10	~ 350	$1.2\times$	iter 3	Stable, optimal by iter 22
20	~ 700	$2.5\times$	iter 2	Stable, optimal by iter 13
40	$\sim 1,400$	$5.0\times$	iter 1	Stable, optimal by iter 2

Buffer/params measured at iteration 10. Network has 2,829 parameters.

Key result: games=5 at $D=4$ reproduces the $D=14$ failure pattern *exactly*.

Threshold: replay buffer $\geq 1.2\times$ network parameters for stable learning.

The Spike-and-Collapse Mechanism



Train reward: stays high
(network fits its training data well)

Eval reward: crashes
(policy does not generalize)

Experiment D: Difficulty Scaling (sims= 25, games= 20)

D	sims/ K	First solve	Outcome	Regime
2	3.12	iter 1	Stable, optimal	Comfortable
3	2.08	iter 3	Stable, optimal	Comfortable
4	1.56	iter 2	Stable, optimal by iter 13	Comfortable
5	1.25	iter 2	Collapsed at iter 26	Fragile
6	1.04	iter 32	Slow bootstrap, then stable	Slow bootstrap

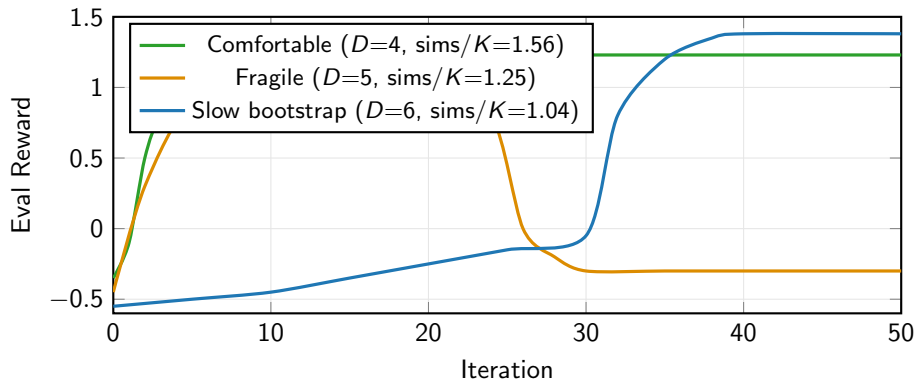
Three distinct regimes emerge as sims/ K decreases:

Comfortable (sims/ $K \geq 1.5$): Fast, stable learning

Fragile (sims/ $K \approx 1.25$): Learns, then collapses – **most dangerous**

Slow bootstrap (sims/ $K \approx 1.0$): Long plateau, then sudden breakthrough

Three Regimes: Conceptual Picture



Paradox: $D=5$ (easier) fails while $D=6$ (harder) succeeds.
 $D=5$ learns fast enough to overshoot into a bad local minimum.
 $D=6$ improves so gradually it never overshoots.

Diagnosing the D=14 Failure

MCTS budget: adequate

- $\text{sims}/K = 120/56 = 2.14$
- Well within “comfortable” regime
- Prior D=14 runs with sims= 120 succeeded

Data volume: **insufficient**

- games= 20, horizon= 135
- $\sim 2,700$ examples/iter
- 27-step plan: each example covers one state
- Effective coverage is thin

Root cause confirmed:

Data starvation,
not a code bug.

Fix: increase
games 20 $\rightarrow$ 50

The spike-and-collapse at D=14 (iter 9 $\rightarrow$ 10) matches the games=5 failure at D=4 exactly.

Summary of Thresholds

Quantity	Threshold	Evidence
MCTS budget	$\text{sims}/K \geq 1.5$	$\text{sims}/K=0.63$ dead, 1.56 stable
Data volume	$\text{buffer} \geq 1.2 \times \text{params}$	games= 5 unstable, games= 10 stable
Difficulty	3 regimes	Comfortable / Fragile / Slow bootstrap

The single most important result:

games=5 at $D=4$ reproduces the $D=14$ failure pattern (spike-and-collapse) at small scale. This confirms **data starvation** – not a code bug – as the root cause.

Lessons and Next Steps

Lessons:

- ① AlphaZero has **sharp phase transitions** in both search budget and data volume
- ② The “fragile” regime is the most dangerous: **looks like success before it collapses**
- ③ Small-scale reproduction of large-scale failures is a powerful diagnostic tool

Next steps:

- ① Rerun $D=14$ with games= 50, sims= 120, iters= 40
- ② Investigate **acceptance gating** as a stabilizer for the fragile regime
- ③ Scale to larger D with budget rules: $\text{sims} \geq 1.5K$, $\text{buffer} \geq 2 \times \text{params}$
- ④ Explore **action masking** to reduce effective action space

Thank You

Questions?

All experiments and code available at:
`AlphaZero_PP/experiments/doors_direct/`